PROJECT REPORT

ON

HOME SERVICE BOOKING

Project Code: CPP34

Enrollment Number: IU2541230474

Table of Contents

1. Project Title
2. Student Details
3. Problem Statement
4. Objectives
5. Features
6. Classes and Responsibilities
7. OOP Concepts Used
8. File Handling
9. UML Diagrams
10. Program Workflow
11. Screenshots
12. Testing
13. Challenges and Solutions
14. Conclusion

1. Project Title

Home Service Booking

2. Student Details

Enrollment Number: IU2541230474

Project Code: CPP34

Student Name: _____

Course / Programme: _____

3. Problem Statement

People often need home services such as plumbing, electrical repair, cleaning and painting. Arranging these services manually can require repeated phone calls, make it difficult to keep track of customer and worker details, and increase the possibility of losing booking information. A small computer-based system can organize service options, customer records, worker details and bookings in one place.

The Home Service Booking project is a menu-driven C++ console application intended to demonstrate how these tasks can be managed using object-oriented programming and file handling. It is an educational prototype; it does not provide live online payments, real-time worker tracking or internet-based booking.

4. Objectives

- Maintain customer information and worker details in structured records.
- Display available home services and their prices.
- Create bookings that connect a customer, service, worker and requested date.
- Search, update and delete booking records through a menu.
- Generate a simple report of stored booking information.
- Save booking data to a file and load it again when required.
- Apply core C++ object-oriented programming concepts in a modular program.
- Validate user input and handle common file-operation or input errors gracefully.

5. Features

- Add customer records with an automatically assigned identifier.
- Add worker records, including the worker's service skill.
- Display stored records and available services.
- Search for a booking using its identifier.
- Update the date of an existing booking.
- Delete a selected booking.

- Book a service by selecting a customer, service, worker and date.
- Display a booking report and calculate/show booking amounts.
- Save and load booking information through the FileManager class.
- Provide a numbered console menu so the user can choose operations.

The exact menu numbering and available actions should be checked against the final executable before submission. The system is intended for demonstration with a limited number of in-memory customer, worker and service records.

6. Classes and Responsibilities

6.1 Person

Person is the common base class for people represented in the system. Its protected data members store an identifier, name and phone number. It provides constructors and a virtual display function so derived classes can show their specific information.

6.2 Customer

Customer inherits from Person and represents a person requesting a home service. It reuses the common identity and contact fields and implements display() to show customer information.

6.3 Worker

Worker inherits from Person and represents a service provider. In addition to the inherited fields, it stores a skill such as Plumbing, Electrical, Cleaning or Painting. Its display() function presents worker details, and getter functions allow the system to retrieve the worker's identifier and name.

6.4 Service

Service represents a service offered by the system. Its private data members include the service identifier, service name and price. Constructors initialize service information, getter functions provide controlled access, and display() presents the service details.

6.5 Booking

Booking stores the details of one booking, including its identifier, customer name, service name, worker name, date and amount. Its constructor initializes the booking, setDate() supports updating the date, and display() presents the booking summary.

6.6 FileManager

FileManager is responsible for file input and output. Its save and load functions write booking data to persistent storage and read saved data back into the application. Separating file operations from menu logic makes the program easier to maintain.

6.7 ServiceSystem

ServiceSystem coordinates the application. It maintains the customer, worker and service collections, manages booking records, displays the menu operations, and calls FileManager when data must be saved or loaded. Its operations include adding records, displaying and searching

records, updating or deleting bookings, booking a service and producing a report. Dynamically allocated arrays are released by the class destructor.

7. OOP Concepts Used

7.1 Classes and Objects

Each class defines the data and operations for a particular concept. For example, a Customer object stores one customer's details, a Service object describes one service, and a Booking object represents one booking. Objects are created and used by ServiceSystem to perform operations.

7.2 Encapsulation

Encapsulation groups data with the functions that operate on it and restricts direct access where appropriate. For example, Service keeps its identifier, name and price private and exposes getter functions. This helps prevent unrelated parts of the program from changing internal values without using the class interface.

7.3 Constructors and Destructor

Default constructors allow objects to be created with initial/default values, while parameterized constructors initialize objects with supplied values. The destructor of ServiceSystem releases dynamically allocated arrays using delete[] so that the memory allocated with new[] is returned when the system object is destroyed.

7.4 Inheritance

Customer and Worker inherit from Person. This avoids repeating shared fields such as identifier, name and phone number. Each derived class adds its own behavior or data; Worker, for example, adds a skill field.

7.5 Runtime Polymorphism

Person declares display() as a virtual function, and Customer and Worker provide their own implementations. When a derived object is accessed through a base-class interface, the virtual function mechanism allows the appropriate derived implementation to be selected at runtime. This demonstrates runtime polymorphism.

7.6 Dynamic Memory

ServiceSystem uses dynamically allocated arrays for customer, worker and service records. Memory is allocated with new[] and released with delete[] in the destructor. Matching allocation and deallocation is important to avoid memory leaks. The fixed array capacity should be checked before inserting more records.

7.7 Modular Programming

The program is divided into header (.h) and implementation (.cpp) files. Header files declare classes and member functions, while implementation files define their behavior. This separation improves readability, reuse and maintainability.

8. File Handling

Booking information is stored in a plain-text data file at Data/bookings.txt. The file contains one booking per line, with fields separated by a delimiter such as the vertical bar character (|). Typical fields include booking ID, customer name, service name, worker name, date and amount. The precise order must remain consistent between saving and loading.

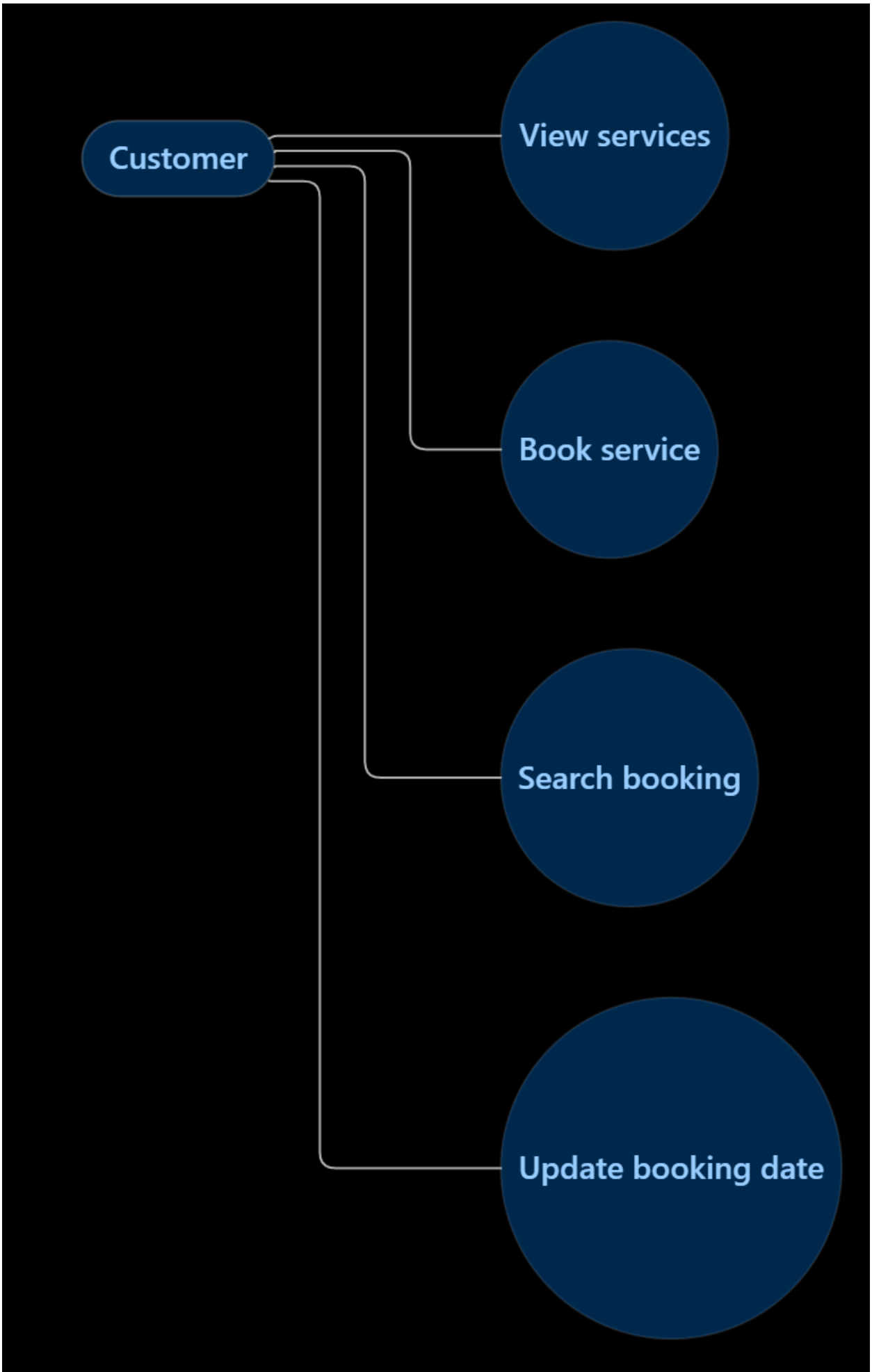
When a booking is created or modified, the application can call FileManager to write the current booking collection to the data file. The load operation reads each line, separates the fields, converts numeric values where required, and reconstructs booking records. File operations should check whether the file opened successfully and handle missing or malformed data without crashing.

Persistence can be tested by creating a booking, saving the records, exiting the program, launching it again, loading the saved data and checking that the booking still appears. The test should be performed from the project root directory so the relative path Data/bookings.txt resolves correctly.

9. UML Diagrams

The following diagrams summarize the system actors, class structure and two key interactions. These diagrams are schematic representations of the intended project design; compare them with the final source code and update them if class names or method signatures have changed.

9.1 Use Case Diagram


Customer
View services
Book service
Search booking
Update booking date

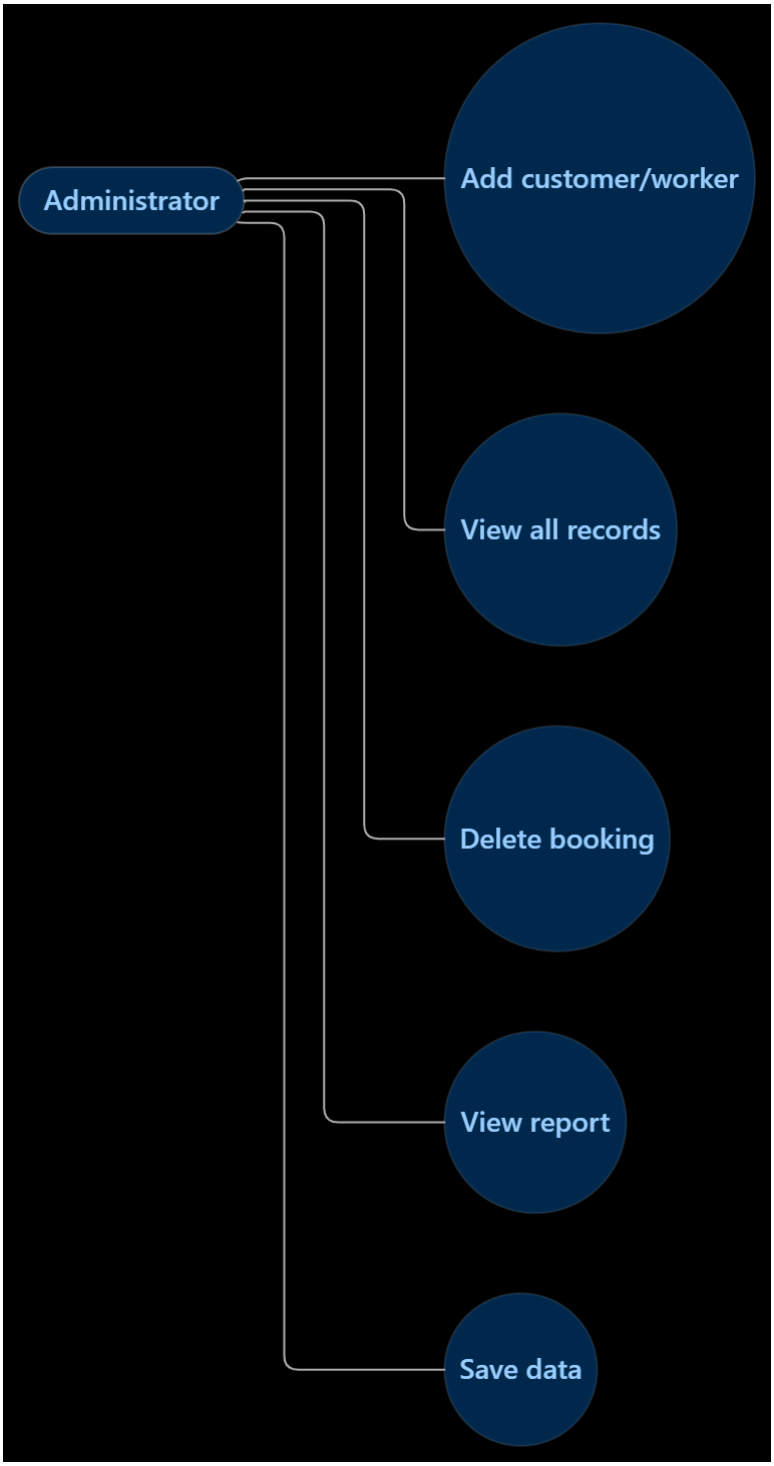


Figure 1. Use cases available to the customer and administrator.

9.2 Class Diagram

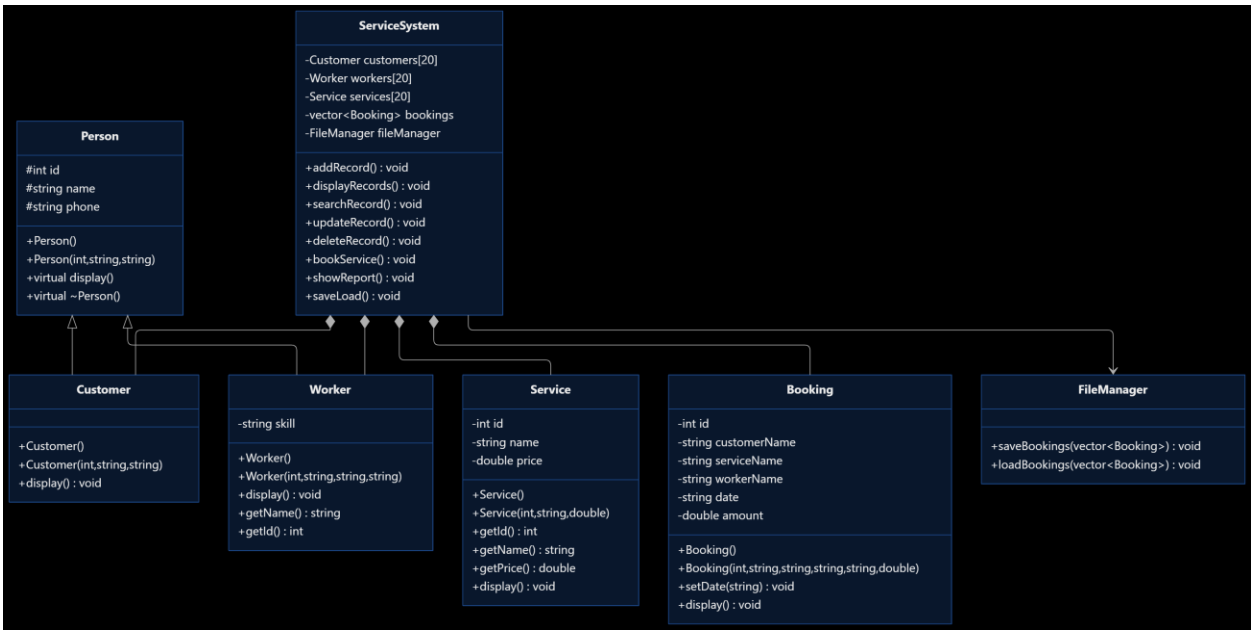


Figure 2. Main classes and their relationships.

9.3 Sequence Diagram 1 — Book a Service

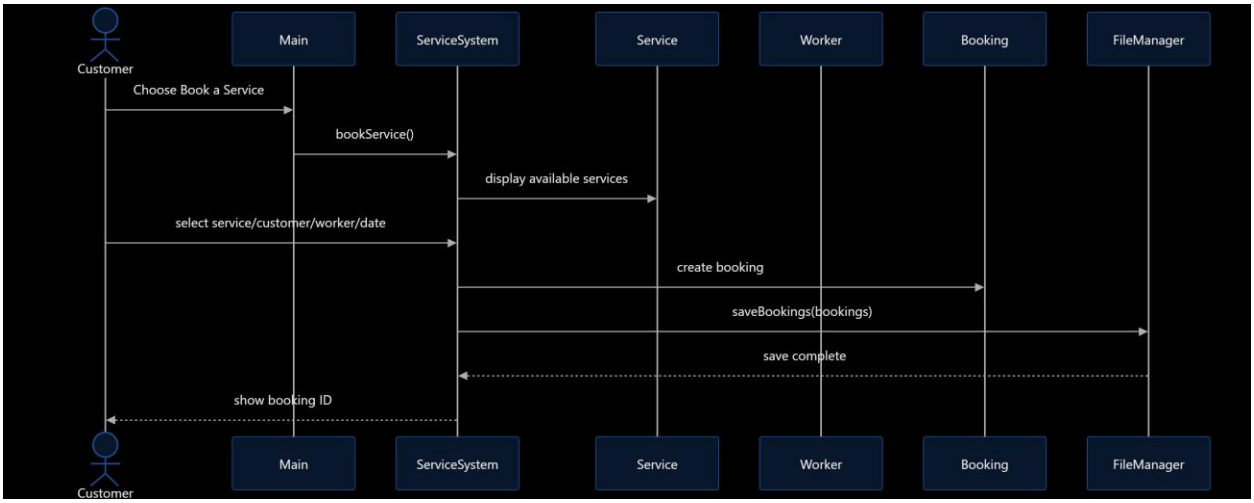


Figure 3. Booking flow from menu selection through saving the booking.

9.4 Sequence Diagram 2 — Update Booking

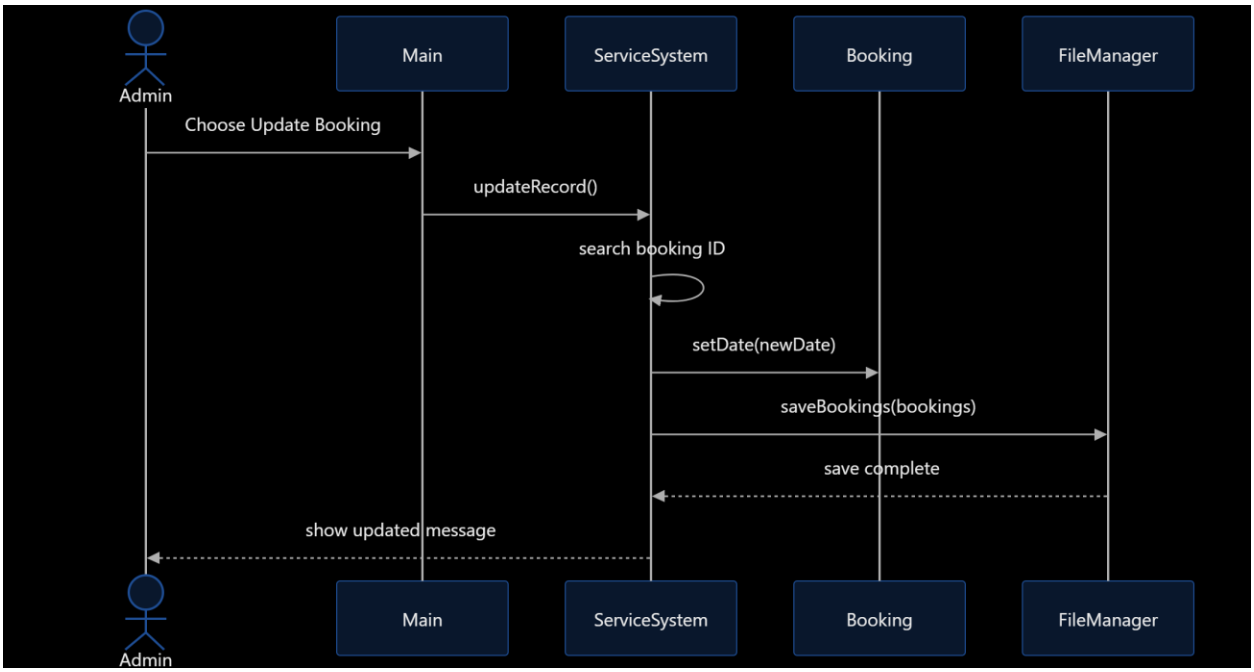


Figure 4. Updating a booking date and saving the revised record.

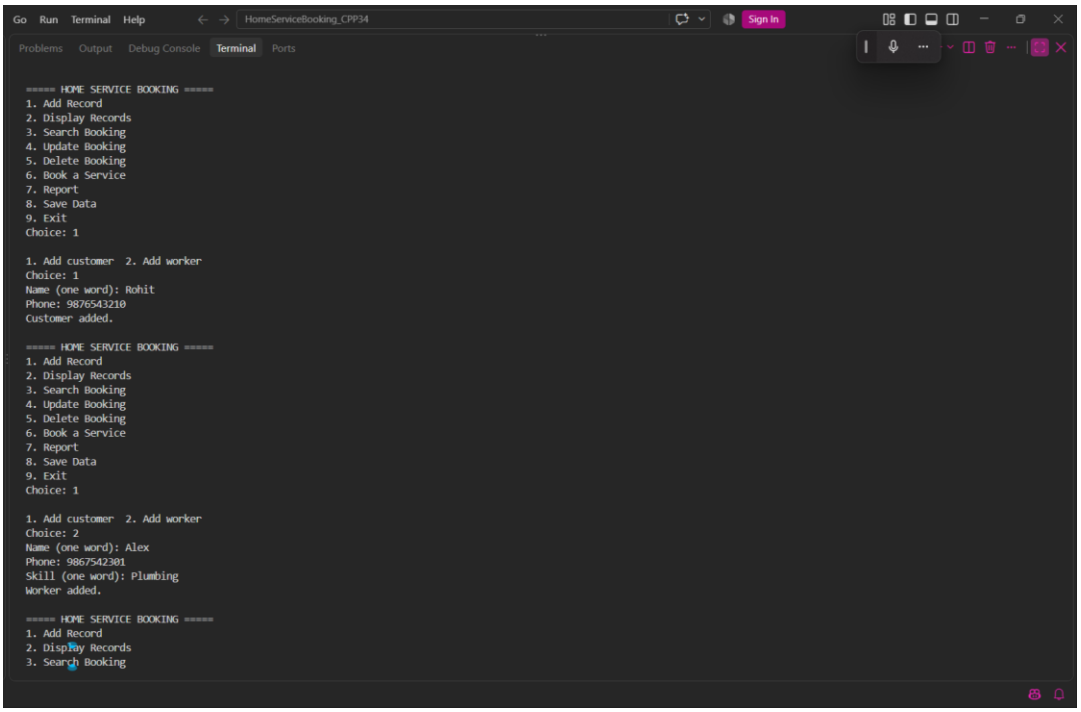
10. Program Workflow

1. The program starts and creates the ServiceSystem object.
2. The application loads or initializes the available service, customer and worker information according to the program setup.
3. A numbered menu is displayed and the user selects an operation.
4. For a booking, the system displays available services and asks for the required customer, worker, service and date details.
5. The program validates identifiers and input values before creating or modifying records.
6. The selected operation is performed: display, search, update, delete, report, or save/load.
7. Booking changes are saved through FileManager when the program's save logic is invoked.
8. The menu is displayed again until the user selects Exit.
9. On termination, the ServiceSystem destructor releases dynamically allocated memory.

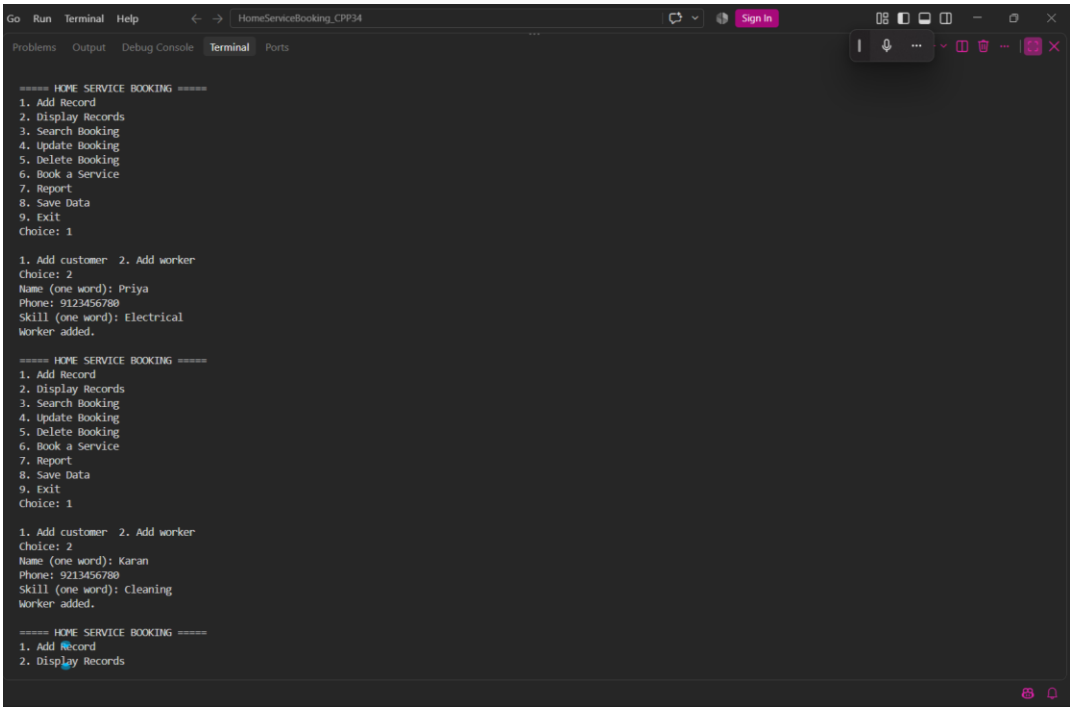
11. Screenshots

Insert screenshots captured from the final compiled program. Each screenshot should be readable and should show the operation being demonstrated. Do not use mock screenshots; capture the actual program running on your computer.

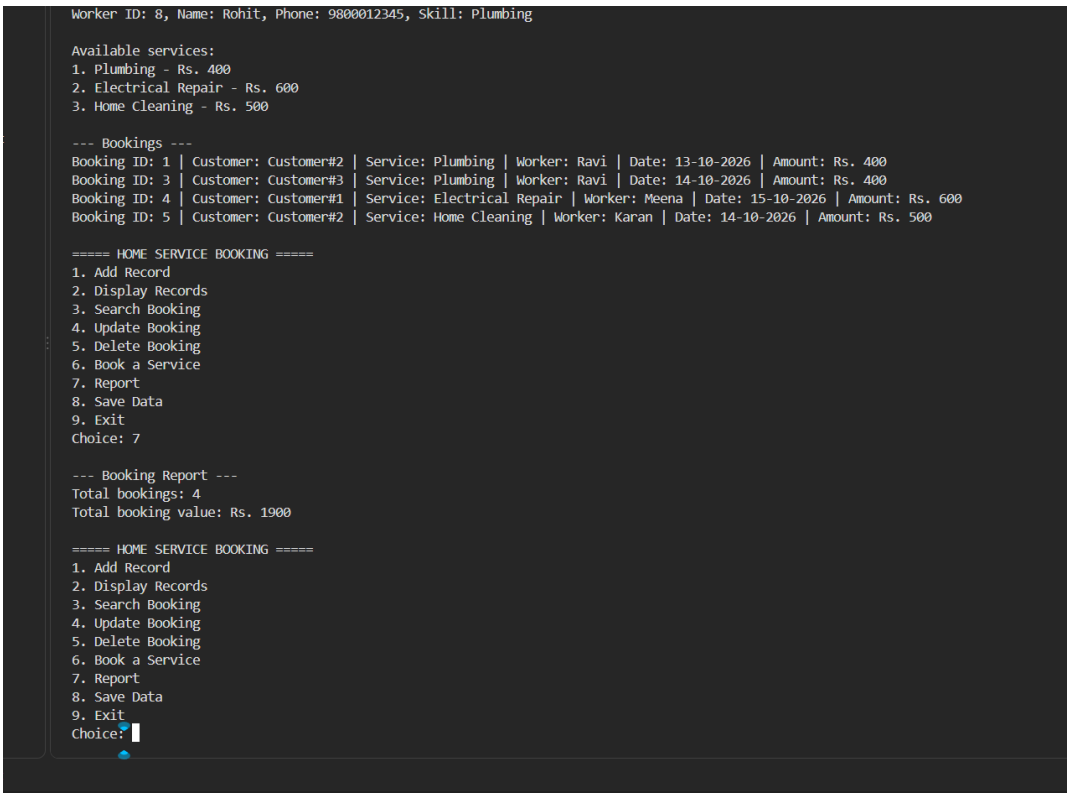
Screenshot 1 — Program startup and main menu



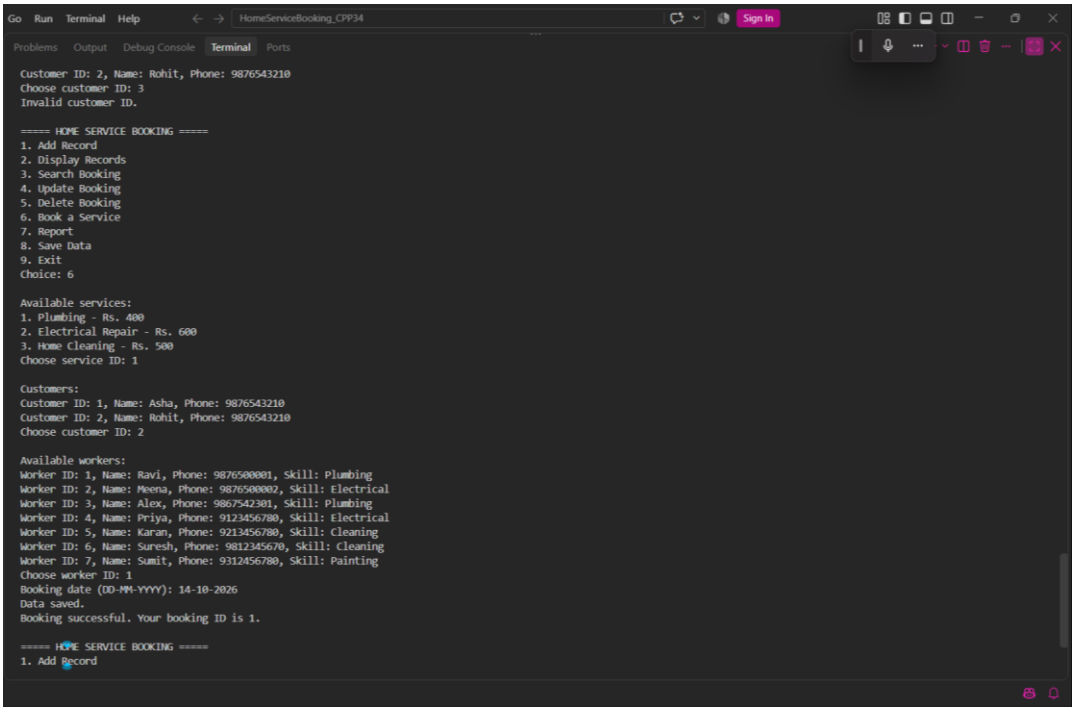
Screenshot 2 — Add customer / worker



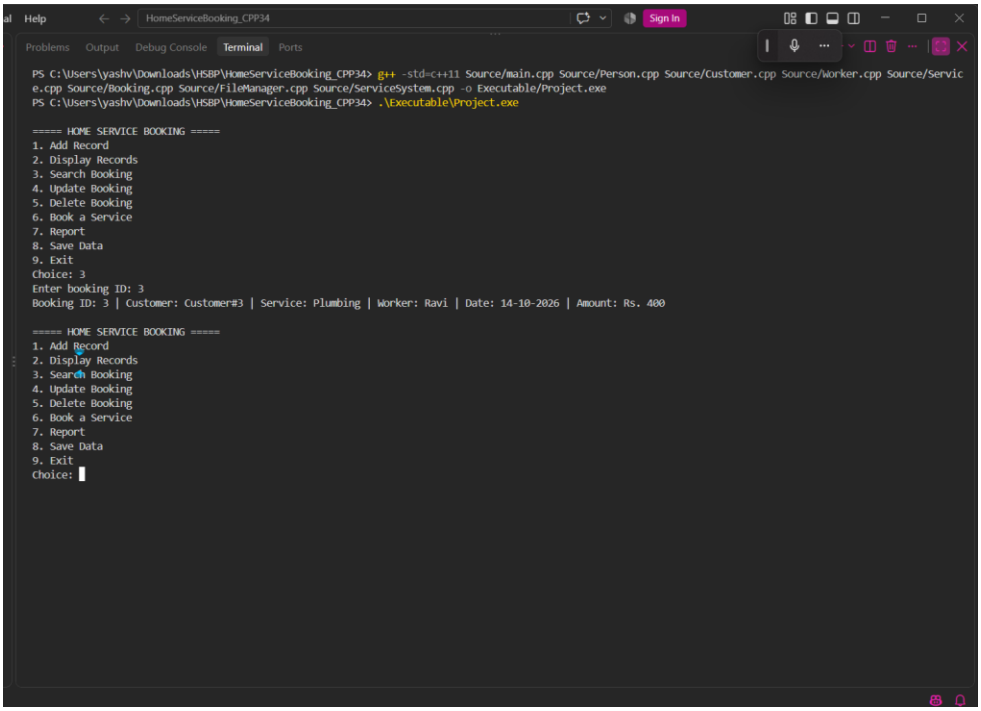
Screenshot 3 — Display services and records

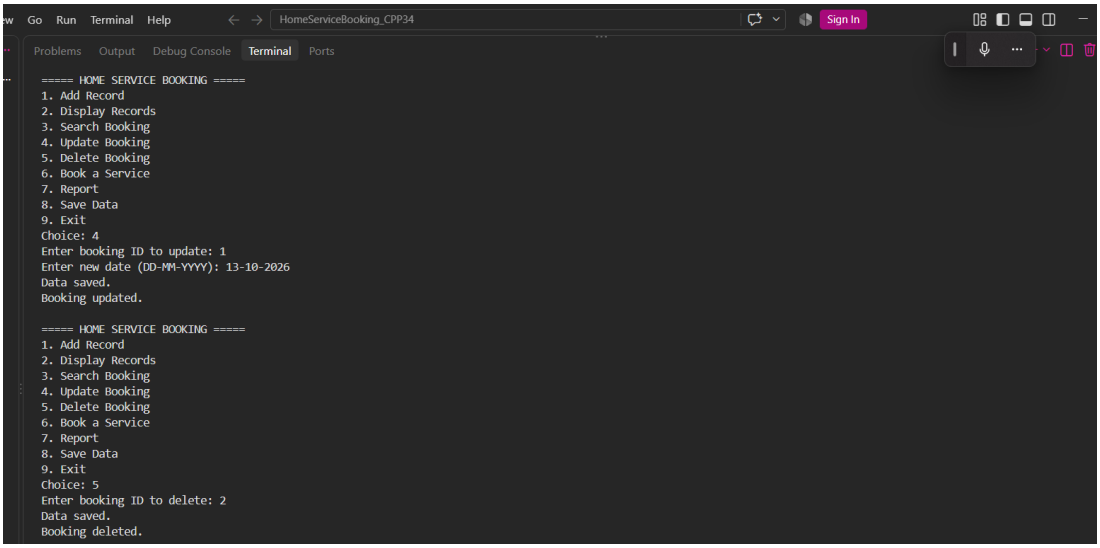


Screenshot 4 — Successful booking



Screenshot 5 — Search or update booking





Screenshot 6 — Save and reload



12. Testing

The following test plan covers typical operations and common invalid inputs. “Actual result” entries below describe the expected behavior to verify during execution; replace them with the observed result from your final program before submission.

Test ID	Input / Action	Expected Output
T01	Start program and select Display	Menu appears; records/services are displayed.
T02	Add a customer with a valid name and 10-digit phone	Customer is stored and assigned an ID.
T03	Add a worker with a skill	Worker is stored and skill is displayed.
T04	Book an existing service with valid customer/worker IDs and date	Booking is created with an ID and amount.
T05	Search using an existing booking ID	Matching booking details are shown.
T06	Search using a nonexistent booking ID	A not-found message is shown; program continues.
T07	Update the date of an	The date changes and

	existing booking	updated data is saved.
T08	Delete an existing booking ID	The selected booking is removed.
T09	Save, exit, restart and reload	Saved booking information remains available.
T10	Enter an invalid menu choice or malformed input	Input is rejected or handled without abnormal termination.

13. Challenges and Solutions

13.1 Organizing related data

Challenge: Customer, worker, service and booking information have different fields and responsibilities. Solution: Separate the information into dedicated classes and use ServiceSystem to coordinate them.

13.2 Avoiding repeated code

Challenge: Customer and Worker share common identity and contact details. Solution: Place shared fields and virtual display behavior in Person, then derive Customer and Worker from it.

13.3 Managing dynamically allocated memory

Challenge: Memory created with new[] must be released correctly. Solution: Provide a destructor that calls delete[] for the allocated arrays, and avoid copying the owning ServiceSystem object unless safe copy behavior is implemented.

13.4 Preserving booking data

Challenge: In-memory data would be lost when the program exits. Solution: Use FileManager and a text file to save and reload booking records, and test persistence by restarting the program.

13.5 Validating input and file data

Challenge: Invalid identifiers, malformed values or an unavailable file can interrupt normal operation. Solution: Check user input, validate selected IDs, verify file-open operations and provide clear error messages. Confirm that the final implementation handles each case during testing.

14. Conclusion

The Home Service Booking project demonstrates a structured approach to managing home-service bookings in a C++ console application. It brings customer, worker, service and booking information together and provides operations for creating, viewing, searching, updating, deleting and reporting records. The use of separate classes, inheritance, encapsulation, constructors, a virtual function, dynamic memory and file handling provides practical experience with core C++ concepts.

The project also highlights the importance of input validation, consistent file formats, memory management and testing. As future enhancements, the prototype could be extended with stronger date validation, more robust file error handling, a database, authentication, a graphical interface and online booking features.

Appendix A — Build and Run

Compile the .cpp source files together using a C++11-compatible compiler. For example, from the project root, compile all implementation files and main.cpp into one executable. Run the executable from the project root so the relative Data/bookings.txt path resolves correctly. Follow the README supplied with the source project for the exact build command.